

Fetch API

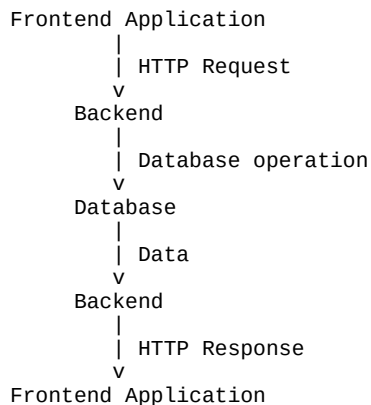
Complete JavaScript Notes — Beginner to Advanced

Topics: `fetch()` basics, GET/POST/PUT/DELETE, Response object, `.json()` parsing, error handling, loading states, CORS, API keys, and network errors.

Learning goal: Understand how a browser application communicates with a backend API and how to build reliable Fetch requests.

1. What is an API?

API stands for **Application Programming Interface**. An API provides a defined way for one software application to communicate with another.



Example: an e-commerce frontend can request `GET /api/products`, and the backend can return product data as JSON.

API vs Protocol

API: defines how applications can interact and what operations are available.

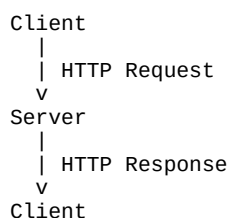
Protocol: defines communication rules. HTTP is a common protocol used by web APIs.

Common API Technologies

Technology	Typical use
HTTP / HTTPS	Web APIs and REST APIs
WebSocket	Real-time, two-way communication
gRPC	High-performance service-to-service communication
GraphQL	Client-controlled data queries; commonly transported over HTTP

2. HTTP Protocol

HTTP stands for HyperText Transfer Protocol. It defines how a client and server exchange requests and responses.



HTTP Request

An HTTP request can contain a method, URL, headers, query parameters, cookies, and a body.

```
POST /api/products HTTP/1.1
Content-Type: application/json
{
```

```
    "name": "Laptop",
    "price": 50000
  }
```

HTTP Response

An HTTP response generally contains a status code, headers, and a body.

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": 1,
  "name": "Laptop",
  "price": 50000
}
```

3. HTTP Methods

Method	Purpose	Typical use
GET	Retrieve data	Get products
POST	Create a resource	Create a product
PUT	Replace/update a resource	Replace product data
PATCH	Partially update	Change only price
DELETE	Delete a resource	Delete a product

Memory aid: GET → Read, POST → Create, PUT → Replace, PATCH → Partial update, DELETE → Delete.

4. What is the Fetch API?

The **Fetch API** provides JavaScript with a way to make HTTP requests. The second argument is optional, and GET is the default method.

```
fetch("https://dummyjson.com/products");

fetch("https://example.com/api/products", {
  method: "GET"
});
```

Why is fetch() asynchronous?

A server may take an unpredictable amount of time to respond. JavaScript can continue executing other code while the request is in progress.

```
console.log("Start");

fetch("https://dummyjson.com/products")
  .then((response) => {
    console.log("Response received");
  });

console.log("End");
```

Typical order: Start → End → Response received.

5. fetch() Returns a Promise

A call to `fetch()` returns a Promise. When an HTTP response is successfully received, the Promise fulfills with a **Response object**.

```
const result = fetch("https://dummyjson.com/products");
console.log(result); // Promise
fetch(url)
  .then((response) => {
    // response is a Response object
  })
  .catch((error) => {
    // handle rejected request
  });
```

6. GET Request

GET is used to retrieve data.

```
fetch("https://dummyjson.com/products")
  .then((response) => {
    console.log(response);
  });
```

GET with JSON parsing

```
fetch("https://dummyjson.com/products")
  .then((response) => response.json())
  .then((data) => {
    console.log(data);
  })
  .catch((error) => {
    console.log(error);
  });
```

7. Response Object

The first `.then()` receives a Response object.

Property / method	Purpose
<code>response.status</code>	HTTP status code, e.g. 200 or 404
<code>response.ok</code>	true for successful 2xx responses; false otherwise
<code>response.statusText</code>	Status text where available
<code>response.headers</code>	Response headers
<code>response.body</code>	ReadableStream for the response body
<code>response.json()</code>	Read and parse JSON response body
<code>response.text()</code>	Read response body as text
<code>response.blob()</code>	Read response body as a Blob

`response.status`

```
fetch(url)
  .then((response) => {
    console.log(response.status);
  });
```

`response.ok`

```
fetch(url)
```

```

    .then((response) => {
      console.log(response.ok);
    });

```

For successful 2xx responses, `response.ok` is true. For 4xx/5xx responses, it is false.

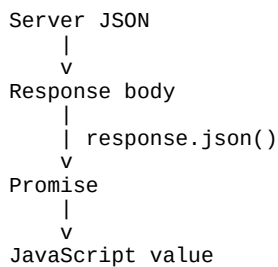
8. response.json()

`response.json()` reads the response body and parses JSON into a JavaScript value. It is asynchronous and returns a Promise.

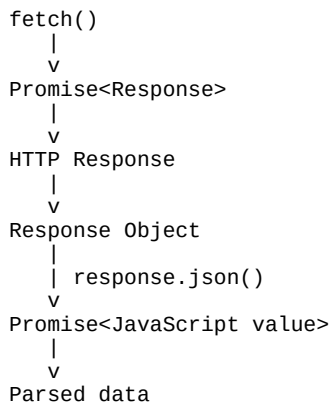
```

fetch("https://dummyjson.com/products")
  .then((response) => {
    return response.json();
  })
  .then((data) => {
    console.log(data);
  });

```



9. Fetch Lifecycle



Teaching model: **fetch** → **Response** → **json()** → **parsed JavaScript value**.

10. Display API Data on a Webpage

```

fetch("https://dummyjson.com/products")
  .then((response) => response.json())
  .then((data) => {
    data.products.forEach((product) => {
      const card = document.createElement("div");
      card.classList.add("product-card");

      const heading = document.createElement("h1");
      heading.textContent = product.title;

      const para = document.createElement("p");
      para.textContent = product.description;

      const subHeading = document.createElement("h3");
      subHeading.textContent = product.brand;

      card.append(heading, para, subHeading);

      const productList =

```

```

        document.querySelector(".product-wrapper");
        productList.append(card);
    });
});

```

Flow: API → fetch() → Response → json() → JavaScript data → DOM manipulation → webpage.

11. HTTP Status Codes

Range	Category	Meaning
100–199	Informational	Informational response
200–299	Success	Request was successful
300–399	Redirection	Further action may be involved
400–499	Client Error	Problem with the request/client
500–599	Server Error	Problem on the server

Memory aid: 1xx → Information, 2xx → Success, 3xx → Redirect, 4xx → Client problem, 5xx → Server problem.

Code	Meaning
200	OK
201	Created
204	No Content
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
500	Internal Server Error
503	Service Unavailable

12. Important: HTTP Errors and fetch()

fetch() normally does **not reject merely because the server returned 4xx or 5xx**. It can fulfill with a Response object.

```

fetch("/api/products/999")
  .then((response) => {
    console.log(response.status); // e.g. 404
  })
  .catch((error) => {
    console.log(error);
  });

```

Therefore, check response.ok when HTTP status errors should be treated as failures.

```

fetch(url)
  .then((response) => {
    if (!response.ok) {
      throw new Error(
        `HTTP Error: ${response.status}`
      );
    }
  });

```

```

    });
  }
  return response.json();
})
.then((data) => console.log(data))
.catch((error) => console.log("Error:", error));

```

13. POST Request

POST is commonly used to create a new resource.

```

const product = {
  name: "iPhone 15 Pro",
  description: "Latest Apple smartphone",
  price: 999.99,
  stock: 50,
  category: "Electronics"
};

fetch("http://localhost:3000/api/products", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify(product)
})
.then((response) => {
  if (!response.ok) {
    throw new Error(`HTTP Error: ${response.status}`);
  }
  return response.json();
})
.then((data) => console.log("Created:", data))
.catch((error) => console.log("Error:", error));

```

Why JSON.stringify()?

It serializes a JavaScript object into JSON text suitable for a JSON request body.

```

JavaScript object
  |
  | JSON.stringify()
  v
JSON string
  |
  v
HTTP request body

```

Why Content-Type?

Content-Type: application/json tells the server that the request body is JSON.

14. PUT Request

PUT is generally used when the client wants to replace or update a resource representation.

```

const product = {
  name: "iPhone 18 Pro",
  description: "Latest Apple smartphone",
  price: 999.99,
  stock: 50,
  category: "Electronics"
};

fetch("http://localhost:3000/api/products/2", {
  method: "PUT",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify(product)
})
.then((response) => {
  if (!response.ok) {

```

```

        throw new Error(`HTTP Error: ${response.status}`);
    }
    return response.json();
})
.then((data) => console.log("Updated:", data))
.catch((error) => console.log("Error:", error));

```

PUT vs PATCH

PUT	Usually replaces the resource representation.
PATCH	Usually applies a partial modification.

```

PATCH /api/products/2

{
  "price": 45000
}

```

15. DELETE Request

DELETE is used to request deletion of a resource.

```

fetch("http://localhost:3000/api/products/3", {
  method: "DELETE"
})
.then((response) => {
  if (!response.ok) {
    throw new Error(`HTTP Error: ${response.status}`);
  }

  if (response.status === 204) {
    return null;
  }

  return response.json();
})
.then((data) => console.log("Deleted:", data))
.catch((error) => console.log("Error:", error));

```

A DELETE response may use 204 No Content. In that case there is no body to parse.

16. Request Components

Component	Purpose	Example
Method	Operation requested	GET, POST, PUT
URL	Resource location	/api/products
Headers	Request metadata	Content-Type, Authorization
Query parameters	Filtering/sorting/pagination	?category=mobile
Route parameters	Identify a resource in path	/products/25
Body	Data sent to server	{"name": "Laptop"}
Cookies	Browser-managed data sent to server	session cookie

Query Parameters

```
GET /api/products?category=mobile&sort=price&order=asc
```

Query parameters are commonly used for filtering, searching, sorting, and pagination.

Route Parameters

```
GET /api/products/25
```

Here, 25 can identify a particular product. Backend frameworks often expose it as a route parameter such as `req.params.id`.

Request Body

```
{
  "name": "Laptop",
  "price": 50000
}
```

The request body contains data being sent to the server and is commonly used with POST, PUT, and PATCH.

17. Error Handling

HTTP Error

The server responded, but the status indicates failure, such as 400, 401, 403, 404, or 500.

Network Error

A network/request failure can occur when the request cannot successfully obtain an HTTP response. Examples include no internet connection, DNS failure, an unreachable server, or certain browser-enforced request blocks.

```
fetch("https://invalid-domain-example.com")
  .then((response) => response.json())
  .catch((error) => {
    console.log("Request failed:", error);
  });
```

HTTP error	Network/request failure
Server responded with an HTTP response	No usable HTTP response is received
Has a status such as 404 or 500	Usually no HTTP status is available to application code
<code>response.ok</code> can be false	<code>fetch()</code> Promise can reject

18. Loading States

Network requests take time. A good UI tells the user that a request is in progress.

```
loading
|
+----> Success
|
+----> Error

const loading = document.querySelector("#loading");
loading.textContent = "Loading...";

fetch("https://dummyjson.com/products")
  .then((response) => {
    if (!response.ok) {
      throw new Error(`HTTP Error: ${response.status}`);
    }
    return response.json();
  })
  .then((data) => {
    console.log(data);
  });
```

```

    })
    .catch((error) => {
      console.log(error);
      loading.textContent = "Unable to load products.";
    })
    .finally(() => {
      loading.textContent = "";
    });

```

Why finally()? It runs after fulfillment or rejection, making it useful for stopping spinners and restoring UI state.

19. CORS

CORS stands for **Cross-Origin Resource Sharing**. It is a browser security mechanism that controls whether JavaScript running on one origin can access a resource from another origin.

What is an origin?

An origin is based on **protocol + host + port**.

```

http://localhost:3000
http://localhost:5000

```

These have different origins because their ports differ.

```

https://example.com
https://api.example.com

```

These have different origins because their hosts differ.

CORS flow

```

Frontend
http://localhost:3000
    |
    | cross-origin request
    v
Backend
http://localhost:5000
    |
    | CORS response headers
    v
Browser

```

A server can send headers such as `Access-Control-Allow-Origin` to indicate which origins are allowed. The browser enforces these rules for frontend JavaScript.

CORS is not authentication

CORS: browser access control for cross-origin requests.

Authentication: determines who or what is allowed to access a protected resource.

20. API Keys

Some APIs require an API key or another credential to identify or authorize a client.

```

fetch("https://api.example.com/data", {
  headers: {
    "X-API-Key": "MY_API_KEY"
  }
});

```

Do not expose secret keys in frontend JavaScript

Code executed in the browser can be inspected by the user. A secret key placed directly in frontend code should therefore be considered exposed.

```
const API_KEY = "secret-value";

fetch("https://api.example.com/data", {
  headers: {
    "Authorization": `Bearer ${API_KEY}`
  }
});
```

Safer architecture



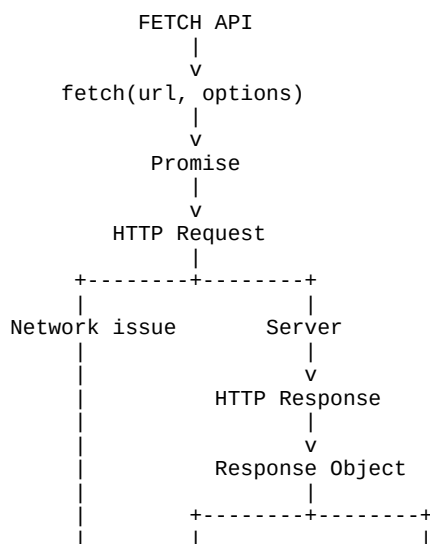
Keep secrets on a trusted server. Server-side environment variables are commonly used to provide secrets to backend applications without embedding them in browser code.

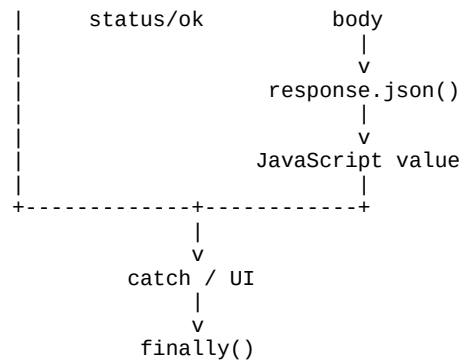
21. CRUD API Example

Operation	Method	Endpoint
Create product	POST	/api/products
Get all products	GET	/api/products
Get one product	GET	/api/products/:id
Replace/update	PUT	/api/products/:id
Partially update	PATCH	/api/products/:id
Delete	DELETE	/api/products/:id

CRUD: Create, Read, Update, Delete.

22. Complete Mental Model





23. Key Points to Remember

1. `fetch()` is used to make HTTP requests.
2. `fetch()` returns a Promise.
3. The first `.then()` receives a Response object.
4. `response.status` gives the HTTP status code.
5. `response.ok` is true for successful 2xx responses.
6. `fetch()` does not normally reject merely because the server returns 4xx or 5xx.
7. Check `response.ok` when HTTP status errors should be treated as failures.
8. `response.json()` reads the response body and parses JSON.
9. `response.json()` also returns a Promise.
10. GET retrieves data; POST commonly creates; PUT commonly replaces; PATCH partially updates; DELETE deletes.
11. Use `JSON.stringify()` when sending a JavaScript object as a JSON request body.
12. Content-Type: `application/json` tells the server the body is JSON.
13. Use loading states to communicate request progress to users.
14. Use `finally()` for cleanup after both success and failure.
15. CORS is a browser security mechanism for cross-origin requests.
16. CORS and authentication are different concepts.
17. Do not expose secret API keys in browser-side JavaScript.
18. Network failures and HTTP errors are different and should be handled appropriately.

24. Recommended Fetch Template

```

const loading = document.querySelector("#loading");

loading.textContent = "Loading...";

fetch("/api/products", {
  method: "GET"
})
.then((response) => {
  if (!response.ok) {
    throw new Error(
      `HTTP Error: ${response.status}`
    );
  }
})

```

```
    });  
  }  
  return response.json();  
})  
.then((data) => {  
  console.log("Data:", data);  
})  
.catch((error) => {  
  console.log("Error:", error);  
})  
.finally(() => {  
  loading.textContent = "";  
});
```

Baseline pattern: start loading → make request → validate HTTP response → parse data → handle success
→ handle failure → stop loading.